

HELEP - Software Architecture Design Process Document

SEN3244 Software Architecture | Part G

Student: FANYI CHARLLSON FANYI

Matricule: ICTU20233841

Date: May 13, 2026

Lecturer: Engr. Tekoh Palma

1. Project Specification

HELEP is an emergency-response platform for citizens, responders, police, and administrators. A citizen signs up, logs in, and can trigger an SOS with location and mode information. The system then assigns a responder, sends notifications, and keeps a police-facing analytics view. The business value is faster emergency response with an auditable incident trail and a workflow that keeps working even if one pod restarts or Kafka temporarily slows down.

2. Requirements Analysis

2.1 Functional requirements (from SRS section 2)

#	Requirement	Source line (SRS section 2)
F1	User registration and login with JWT authentication	Fill from your SRS
F2	Authenticated citizens can trigger an SOS with latitude, longitude, and mode	Fill from your SRS
F3	Citizens can cancel an active SOS	Fill from your SRS
F4	System assigns a responder and supports confirm / release events	Fill from your SRS
F5	System detects zone events and emits alerts	Fill from your SRS
F6	System delivers simulated SMS/push notifications	Fill from your SRS
F7	Police can query stats and summaries	Fill from your SRS
F8	Health, readiness, and metrics endpoints support operations	Fill from your SRS

2.2 Non-functional requirements

NFR	Measurable acceptance criterion
Availability	A single pod restart must not stop the SOS workflow; requests continue while other pods recover.

NFR	Measurable acceptance criterion
Usability	A citizen can sign up and trigger SOS with a small number of REST calls and JSON payloads.
Confidentiality	JWT secrets stay in Kubernetes Secrets and protected routes reject missing or invalid bearer tokens.
Integrity	One incident maps to one assignment record, and duplicate responder claims are rejected.
Reliability	At-least-once Kafka delivery plus idempotent handlers prevent lost incidents under pod restarts.
Scalability	The dispatch and analytics consumers scale horizontally and process multiple partitions in parallel.
Compatibility	The solution deploys on Minikube, k3s, or any conforming Kubernetes cluster using Helm values.

2.3 Constraints (SRS section 4)

Constraint	Architectural risk
Single responder at any moment	Atomic claim logic is required in dispatch-service; otherwise two pods may reserve the same responder.
Trigger to notify must be fast	Synchronous fan-out would violate the response-time target, so the design uses Kafka choreography.
Local dev should be lightweight	SQLite is simple but introduces file-based persistence limits and future migration cost.
Kubernetes submission must be portable	Hard-coded image tags, hosts, or ports would reduce portability across clusters.

3. Architectural Drivers and ASRs (Lecture 1 material)

The three strongest architectural drivers are availability, reliability, and scalability.

Quality attribute	Why it matters in HELEP	Design mechanism
Availability	Emergency events cannot wait for manual recovery	Event-driven flow, pod restarts, readiness probes
Reliability	Incident handling must not lose state or reorder key events	Kafka manual commit, idempotent handlers, unique assignment rows
Scalability	Dispatch and analytics must absorb bursts	Partitioned topics and consumer groups

4. Component Identification (Lecture 4 step)

4.1 SRS-listed components

The SRS components that matter here are: User Management, Emergency Component, Incident Report and Response, Localization, Alert Management, Alert Delivery, Feedback and Review, and Analytics and Statistics.

4.2 Your service decomposition

SRS component	HELEP service	Justification
User Management	user-service	Identity, signup, login, JWT, contacts are cohesive in one bounded context.
Emergency Component	sos-service	Trigger/cancel incident lifecycle belongs to one command-side service.
Incident Report and Response	dispatch-service	Assignment, release, and responder confirmation are one workflow.
Localization	dispatch-service	Distance calculation and zone logic are part of assignment.
Alert Management	dispatch-service + notification-service	Decision logic stays in dispatch; delivery stays in notification.
Alert Delivery	notification-service	Delivery is a separate consumer responsibility.
Feedback and Review	Out of scope	Not required for the 24-hour build.
Analytics and Statistics	analytics-service	Read-only police projections and summaries fit a CQRS read model.

5. Architectural Style - Choice and Justification (Lecture 2)

HELEP uses event-driven microservices. The user-facing REST endpoints are thin command APIs, while Kafka carries the workflow between services. That choice fits the top ASRs better than a synchronous design because it reduces coupling, keeps response time low, and lets the system replay events after a crash.

Alternatives considered

Alternative	Could it satisfy the top ASRs?	Dominant trade-off
Monolith	It could satisfy the functional flow, but scaling and fault isolation would be weak.	Simplicity versus resilience.
Synchronous SOA	It could work for a small prototype, but latency chains and cascading failures would hurt availability.	Clear HTTP calls versus fragility.
Serverless	It could scale, but stateful SQLite and broker-driven choreography are a poor fit.	Elasticity versus stateful complexity.

6. Architectural Patterns Applied (Lecture 3 material)

Pattern	Where it appears	What problem it solves
Choreographed Saga	services/sos-service/app/main.py, services/dispatch-service/app/main.py, services/notification-service/app/main.py	Coordinates SOS, dispatch, and notification without a central orchestrator.
Pub/Sub	services/*/app/events.py	Decouples producers from consumers through Kafka topics.
Repository	services/*/app/db.py	Hides SQLite behind service-level persistence functions.
Strategy	services/dispatch-service/app/matching.py	Lets dispatch switch matching algorithms without changing the caller.
Outbox-lite	services/sos-service/app/main.py	Reduces the risk of losing an event between DB write and Kafka publish.
Circuit breaker	services/*/app/events.py	Prevents repeated retries against a failing broker.
CQRS	services/analytics-service/app/main.py	Separates command handling from the read-only analytics model.
Sidecar-like observability	charts/helep/charts/*/templates/service.yaml	Exposes metrics for Prometheus without polluting business logic.

Diagram 1: Runtime flow

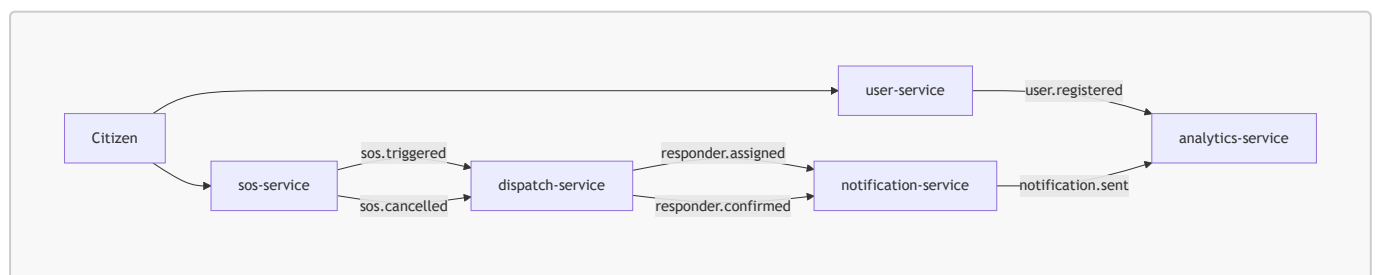


Diagram 2: Service decomposition

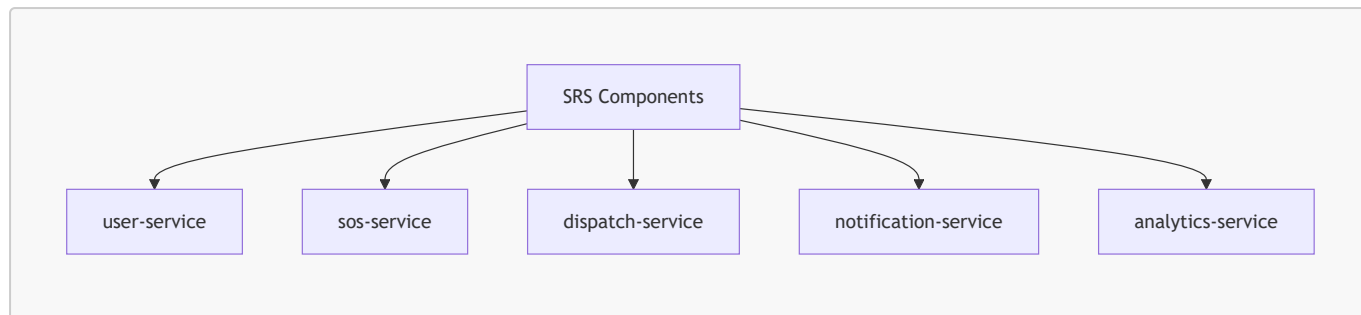
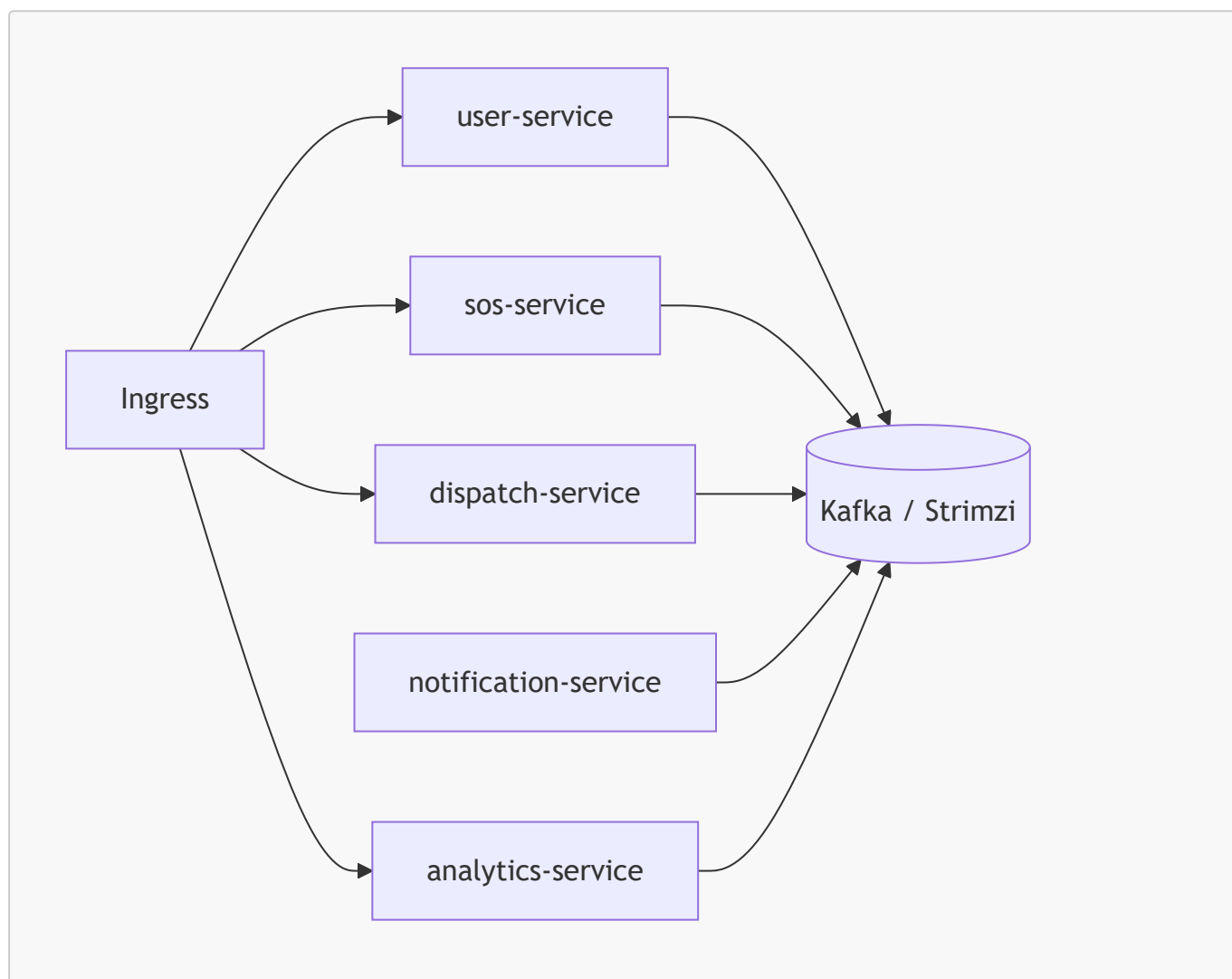


Diagram 3: Deployment view



7. Architecture Decision Records (ADRs)

ADR-001: Kafka partition key is incident_id

Context: One SOS must keep its events ordered.

Decision: Use incident_id as the Kafka key.

Consequences: All events for one incident land on the same partition, which preserves order. The downside is that a hot incident can concentrate load on one partition.

Alternatives considered: Random keys would improve spread but break ordering.

ADR-002: SQLite per service instead of shared Postgres

Context: The project needed a lightweight, easy-to-run local stack.

Decision: Keep one SQLite database per service.

Consequences: Each service owns its data and the repository pattern stays simple. The downside is that SQLite is not the final production database choice.

Alternatives considered: Shared Postgres would be more production-like but would increase coupling.

ADR-003: Helm umbrella chart instead of separate charts

Context: The five services must deploy together with shared configuration.

Decision: Use one umbrella chart with service subcharts.

Consequences: The app can be deployed consistently from one entry point, but values must be managed carefully to avoid drift.

Alternatives considered: Separate charts would add flexibility but make coordinated deployment harder.

8. Trade-offs and Improvement Perspectives

The three weakest points of the current architecture are SQLite per service, single-broker development Kafka, and operational complexity.

Weak point	Improvement
SQLite per service	Migrate to PostgreSQL or stronger persistent volumes with backup strategy.
Single-broker Kafka	Use a 3-broker Strimzi cluster with replication factor 3 and stronger quorum settings.
Operational complexity	Add deployment automation, log correlation IDs, and clearer health dashboards.

9. Submission Checklist

- ☒ Every section above completed
- ☒ At least 3 diagrams included
- ☒ Every choice traced to an SRS line, NFR, or ASR
- ☒ 3 ADRs included
- ☒ Word count around 2000-3000